In chapter 3 we have described balanced trees, which offer the best compromise when it comes to containers, and are ideal to efficiently store dynamically-changing data on which we need to perform frequent searches: in appendix C we have discussed and compared the options we have for containers providing fast lookup, fast insertion or fast removal, and trees offer the best tradeoff between all the operations.

Balanced trees, in particular, guarantee logarithmic running time in the worst-case for all the main operations.

In the general case, when we don't know anything about the data we need to store and (later) search, this is really the best we can hope.

But... what happens when we know that we will only store certain types of data in a container? Turns that there are several cases where if we have more information on the kind of data we need to handle, we can leverage better algorithms than the general-purpose ones.

Take, for instance, sorting: if we know that keys are integers in a limited range, we can use *RadixSort* or *BucketSort*, which means achieving sub-linearithmic performance, defying the lower bound for sorting by comparison[3].

Likewise, if we know that we need to sort strings, then there exist several specialized algorithms, like *3-way string quicksort*, that are optimized for this kind of data and perform better than plain, general-purpose quicksort (or any comparison-based sorting algorithm).